NAVIGATION CALCULATOR

Theoretical framework, governing formulas, numerical workflow, and implementation.

1. Scope and purpose

This program evaluates **Under Keel Clearance (UKC)** for a ship transiting in shallow, restricted, or canal-type waters by combining, in a single operational calculation chain, the following effects:

1. **hydrodynamic squat** generated by the ship moving through finite-depth and laterally confined water;
2. **draft increase due to density difference**, represented as a freshwater-type allowance;
3. **dynamic heel allowance**, including both wind-induced heel and turn-induced heel, converted into an equivalent vertical bilge penetration;
4. **wave-induced vertical motion allowance**, represented by three simplified methods and combined into a single adopted wave allowance.

The script is therefore not a single-formula squat calculator. It is a **compound vertical-clearance model** whose final output is a net seabed clearance after subtracting multiple dynamic allowances from the nominal water depth.

At conceptual level, the calculator answers the following engineering question:

> Given a ship of known principal dimensions, loading condition, speed, channel geometry, wind, turning condition, and wave climate, what is the remaining safe vertical clearance after accounting for the main operational sinkage and motion penalties?

The implementation is deliberately practical and operational. It does not attempt a full CFD, a full 6-DOF time-domain seakeeping simulation, or a ship-specific hydrostatic reconstruction from loading computer data. Instead, it assembles a set of widely used empirical and semi-empirical formulations into a single deterministic workflow suitable for quick assessment, comparison of alternatives, and preliminary navigation planning.

---

2. What the calculator actually computes

The script computes a gross geometric clearance first and then subtracts dynamic penalties.

The **gross under keel clearance** is:

$$UKC_{gross} = h - T$$

where:

- h = nominal water depth;
- T = static draft of the ship.

The script then computes a total dynamic vertical allowance:

$$\Delta Z_{total} = S_{adopted} + \Delta T_{\rho} + Z_{heel} + Z_{wave}$$

where:

- $S_{adopted}$ = adopted hydrodynamic squat;
- ΔT_{ρ} = density-related draft penalty;
- Z_{heel} = heel-induced bilge penetration allowance;
- Z_{wave} = adopted wave-induced vertical motion allowance.

The **net under keel clearance** is then:

$$UKC_{net} = UKC_{gross} - \Delta Z_{total}$$

which is equivalent to:

$$UKC_{net} = (h - T) - \left(S_{adopted} + \Delta T_{\rho} + Z_{heel} + Z_{wave}\right)$$

The script also defines an **effective available depth**:

$$D_{eff} = h - \Delta Z_{total}$$

and a **depth-to-draft safety ratio**:

$$R_{DT} = \frac{D_{eff}}{T}$$

The operational acceptance criterion embedded in the program is:

$$\frac{D_{eff}}{T} \geq 1.10$$

This ratio is central to the program logic. If the condition is not met, the code launches an internal search for a greater nominal depth until the criterion is satisfied.

---

3. General modelling philosophy

The calculator is structured around a conservative screening philosophy rather than a single predictive doctrine.

3.1 Multiple squat methods instead of one

Hydrodynamic squat is not predicted by one equation only. The code evaluates six different methods:

- ICORELS (1980)
- Römisch (1989)
- Millward (1992)
- Eryuzlu (1994)
- Barrass (2004)
- Ankudinov / MARSIM (2009)

The script sorts the six squat values from largest to smallest and adopts:

$$S_{adopted} = \frac{S_{(1)} + S_{(2)} + S_{(3)}}{3}$$

where $S_{(1)} \geq S_{(2)} \geq S_{(3)}$ are the three largest squat predictions.

This is not the maximum of all methods, nor the average of all methods. It is the **mean of the three highest values**, which is an explicitly conservative compromise between a pure envelope approach and a central-average approach.

3.2 Additive combination of allowances

Once the adopted squat is obtained, all remaining penalties are simply added:

$$\Delta Z_{total} = S_{adopted} + \Delta T_{\rho} + Z_{heel} + Z_{wave}$$

This means the model is **deterministic and additive**. The implementation does not account for statistical dependence between squat, wind heel, and wave motion, and it does not perform phase-resolved combination. In engineering terms, it assumes that these allowances must all be reserved simultaneously in the vertical clearance budget.

3.3 Operational rather than vessel-specific hydrostatics

The code uses ship-type averages for several geometric and hydrostatic quantities. It does not ask for a full hydrostatic book. Instead, it infers quantities such as block coefficient, metacentric height ratio, bilge-keel factor, and windage scaling parameters from the selected vessel category.

This makes the calculator fast and convenient, but it also means that the result is best understood as a **planning-level estimate** rather than a final class-approved loading condition assessment.

3.4 Historical development behind the adopted methodology

The logic of the calculator becomes clearer when read against the historical development of squat and UKC practice.

Early squat research was driven by the need to understand why ships in shallow water or confined channels appeared to sit lower and trim differently while underway. Constantine (1960) examined subcritical, critical, and supercritical behaviour, and Tuck (1966) provided one of the landmark analytical treatments for slender bodies in shallow water. Tuck's work established the importance of the **depth Froude number** and clarified why sinkage and trim become highly sensitive as critical conditions are approached.

Beck, Newman, and Tuck (1975) then extended this hydrodynamic line of thought to dredged channels, showing that lateral confinement and exterior shallow regions can materially change the pressure field, ship sinkage, trim, and resistance. This matters here because the calculator is not a pure open-water tool: it explicitly distinguishes unrestricted, restricted, and canal-type sections.

As channel design practice became more operational, empirical and semi-empirical formulas gained importance because they could be applied quickly with limited input data. PIANC (1997) consolidated a broad family of squat methods for design use, and later work by Briggs (2006, 2011) reinforced a practical lesson that is central to this script: **no single squat formula is universally best for all ship types and all channel configurations**. That historical lesson explains why this calculator deliberately evaluates several methods and then adopts a conservative combined value rather than trusting one equation blindly.

The Ankudinov / MARSIM line of development, beginning with Ankudinov and Jakobsen (1996), Ankudinov et al. (1996, 2000), and later documentation by Briggs (2009), added a more composite interpretation in which squat is treated as the result of both **midship sinkage** and **dynamic trim**, with correction terms for channel restriction and ship features. That perspective is reflected directly in the implemented Ankudinov block.

The broader UKC philosophy is equally historical. Modern fairway design does not treat squat as the only clearance loss. Guidance developed in practice and later summarised in PIANC-type design frameworks treats static draught uncertainty, water-density change, heel, and wave allowance as distinct components of a total clearance budget. The present calculator follows that operational tradition: hydrodynamic squat is only one term inside a larger UKC summation.

This historical progression explains the structure of the script. The code moves from classic ship-squat hydrodynamics, to empirical design formulas, to a modern multi-allowance UKC budget that is suitable for rapid navigation assessment.

---

4. Units, constants, and conventions

The script is fundamentally written in SI units, with one notable exception.

4.1 Main units

The program uses:

- length in metres;
- area in square metres;
- volume in cubic metres;
- mass in metric tonnes where entered by the user or inferred from type tables;
- speed in metres per second for most hydrodynamic calculations;
- angles in degrees at input level and radians inside trigonometric operations.

4.2 Knot conversion used by Barrass

The Barrass formulation in the script uses speed in knots. Therefore the code converts ship speed from metres per second to knots as:

$$V_k = 1.94384\, V_s$$

where:

- V_s = ship speed in m/s;
- V_k = ship speed in kn.

4.3 Gravitational acceleration

The code sets:

$$g = 9.80665 \text{ m/s}^2$$

4.4 Air and water constants used in heel calculation

For wind-induced heel, the script adopts:

$$\rho_a = 1.25 \text{ kg/m}^3$$

and

$$\gamma_w = 10052 \text{ N/m}^3$$

where ρ_a is air density and γ_w is the specific weight of seawater used in the restoring moment calculation.

4.5 Sign and interpretation conventions

All allowances are treated as **positive losses of vertical clearance**. The code therefore does not carry explicit positive/negative sign conventions for heave, trim, or roll direction in the final UKC budget. Instead, it converts all governing quantities into positive magnitudes representing required clearance reservation.

---

5. Inputs required by the program

The GUI and the console path gather four broad groups of inputs.

5.1 Ship particulars

The user defines or selects:

- ship type;
- length between perpendiculars L_{pp};
- beam B;
- draft T;
- tonnage-related value used by the internal ship-type tables;
- loading percentage.

The ship type drives several embedded coefficients, including approximate values for:

- block coefficient C_B;
- ratio GM/T;
- bilge-keel or bilge-geometry factor F_K;
- windage regression parameters for frontal and lateral exposed areas.

5.2 Channel and confinement inputs

The user provides:

- water depth h;
- channel type: `unrestricted`, `restricted`, or `canal`;
- bottom width W when relevant;
- trench height parameter h_T;
- bank slope parameter n.

These variables define the channel cross-sectional area, top width, hydraulic depth proxies, and blockage factor.

5.3 Manoeuvring and wind inputs

The user provides:

- relative wind speed V_{WR};

- relative wind angle θ_{WR};
- turning index entered as K_R;
- rudder angle δ_R;
- transient heel coefficient C_ϕ.

These values are used in the wind and turning heel submodels.

5.4 Wave inputs

The program also requires:

- significant wave height H_s;
- wave period T_w;
- wave encounter/incidence angle ψ;
- roll amplification factor μ;
- effective wave slope coefficient γ;
- number of waves encountered N_w;
- exceedance probability P_m.

These variables drive the three simplified wave allowance methods.

---

6. Internal reconstruction of ship properties

Because the script is intended for rapid operational assessment, it reconstructs several ship properties from reduced information.

6.1 Block coefficient and displacement volume

A central quantity is the displacement volume:

$$\nabla = C_B \, L_{pp} \, B \, T$$

This is the standard relation between actual displaced volume and the rectangular reference block $L_{pp}BT$.

From a physical point of view:

- a lower C_B corresponds to a finer ship form;
- a higher C_B corresponds to a fuller ship form;
- fuller ships generally induce stronger shallow-water pressure-field effects and therefore can exhibit stronger squat responses.

6.2 Waterplane coefficient

The script estimates the waterplane coefficient through:

$$C_{WP} = 0.18 + 0.86\, C_B$$

This coefficient is later used in the density-penalty expression. A larger waterplane coefficient implies a larger waterplane area for a given principal geometry, which generally reduces immersion change for a given change in buoyant support.

6.3 Metacentric height

The code does not ask for a measured GM. It reconstructs it from a ship-type ratio:

$$GM = \left(\frac{GM}{T}\right)_{type} T$$

This is a practical approximation. In a real loading condition, GM may differ significantly depending on loading, ballast distribution, free surface, and trim. In the calculator, however, the value is intentionally simplified to maintain usability.

6.4 Vertical centres and transverse hydrostatic approximations

The code approximates:

$$KB = T\left(0.84 - \frac{0.33C_B}{0.18 + 0.87C_B}\right)$$

$$BM = \frac{B^2}{20.4\, C_B\, T}$$

$$KG = KB - GM + BM$$

These formulas provide a quick internal estimate of the transverse hydrostatic chain.

Interpretation:

- KB approximates the vertical location of the centre of buoyancy above keel;
- BM is the transverse metacentric radius;
- KG is the vertical location of the centre of gravity implied by the chosen GM.

The calculator uses these values not for a full intact stability analysis, but specifically to estimate restoring lever scale and turn-induced heel response.

6.5 Windage areas

The program estimates frontal and lateral windage areas via power-law regressions of the form:

$$A_{VF} = a_f \, \text{tonnage}^{b_f}$$

$$A_{VL} = a_l \, \text{tonnage}^{b_l}$$

where the coefficients a_f, b_f, a_l, and b_l depend on ship type.

The code also defines:

$$x_L = \frac{L_{pp}}{2}$$

and

$$KG_W = T + \frac{A_{VL}}{2L_{pp}}$$

Here:

- x_L is a simplified longitudinal centre of lateral wind area;
- KG_W is an approximate vertical position of the lateral wind force application point.

These are not detailed aerodynamic centroids obtained from a ship-specific profile. They are expedient surrogates used by the wind-heel routine.

---

7. Channel geometry and blockage modelling

The confinement model is one of the most important parts of the calculator because several squat formulas depend strongly on blockage and hydraulic depth.

7.1 Midship immersed area

The script approximates midship immersed area as:

$$A_S = 0.98 \, BT$$

The factor 0.98 is a practical correction that slightly reduces the purely rectangular estimate BT.

7.2 Unrestricted water treatment

For `unrestricted` conditions, the script does **not** use an infinite width. Instead, it introduces an effective width:

$$W_{eff} = \left(\frac{7.04}{C_B^{0.85}} \right) B$$

Then it defines a notional channel area:

$$A_C = W_{eff}\, h$$

and top width:

$$W_{Top} = W_{eff}$$

The blockage factor becomes:

$$S = \frac{A_S}{A_C}$$

This is a notable implementation choice. Even in unrestricted water, the script assumes that the ship influences only a finite lateral flow domain rather than the entire sea width.

7.3 Restricted and canal sections

For `restricted` and `canal` cases, the code models the channel cross-section as trapezoidal:

$$A_C = W\, h + n\, h^2$$

$$W_{Top} = W + 2nh$$

and again:

$$S = \frac{A_S}{A_C}$$

where:

- W = bottom width;
- n = side-slope parameter;
- h = water depth.

The blockage factor S is dimensionless. As S increases, confinement increases and squat predictions generally rise.

7.4 Hydraulic interpretation of blockage

The blockage factor is essentially the ratio between the ship's immersed cross-sectional area and the available water cross-sectional area:

$$S = \frac{A_S}{A_C}$$

A small S indicates relatively unconstrained flow around the hull. A larger S indicates that the same displacement of water must pass through a smaller available cross-section, increasing velocity around the hull and lowering pressure under Bernoulli-type reasoning. The result is stronger sinkage and trim effects.

---

8. Froude numbers used by the script

The program uses two Froude numbers.

8.1 Depth Froude number

$$F_{nh} = \frac{V_s}{\sqrt{gh}}$$

This compares ship speed to shallow-water wave celerity based on total depth. It is one of the most important variables in squat analysis.

As F_{nh} approaches unity, the flow approaches critical depth-based conditions and many empirical formulas become highly sensitive.

8.2 Draft Froude number

$$F_{nT} = \frac{V_s}{\sqrt{gT}}$$

This number compares speed to a draft-based velocity scale and appears in the Eryuzlu formulation.

---

9. Hydrodynamic squat model set

The code evaluates six different squat formulas. Because the script is meant to document the actual implementation, the formulas below are described exactly in the form effectively used in `navigation-calculator.py`.

9.1 ICORELS (1980)

This is one of the classic shallow-water squat formulations and is widely used in concept-stage assessment.

The script first selects an empirical coefficient C_S:

- if ship type is `container`, $C_S = 2.0$;
- else if $C_B < 0.70$, $C_S = 1.7$;
- else if $0.70 \leq C_B < 0.80$, $C_S = 2.0$;
- else $C_S = 2.4$.

The formula then used is:

$$S_{ICORELS} = C_S \left(\frac{\nabla}{L_{pp}^2} \right) \left(\frac{F_{nh}^2}{\sqrt{1 - F_{nh}^2}} \right)$$

In the script, the denominator is guarded numerically as:

$$\sqrt{\max(0.001, 1 - F_{nh}^2)}$$

so that the calculator does not fail when F_{nh} approaches or exceeds values producing a non-positive expression under the square root.

Physical meaning

The ICORELS expression increases with:

- displacement volume;
- ship speed;
- reduced depth.

It is structurally a Bernoulli-type sinkage law moderated by a criticality term.

9.2 Römisch (1989)

The Römisch implementation in the script is more elaborate because it distinguishes between unrestricted, canal, and restricted geometry through different celerity scales and correction factors.

Step 1: celerity scales

The code defines:

$$C_U = \sqrt{gh}$$

$$h_m = \frac{A_C}{W_{Top}}$$

$$C_m = \sqrt{gh_m}$$

$$h_{mT} = h - \left(\frac{h_T}{h}\right)(h - h_m)$$

$$C_{mT} = \sqrt{gh_{mT}}$$

These represent different hydraulic depth-based celerities depending on how confinement is idealised.

Step 2: correction factors

The script computes:

$$K_U = 0.58\left(\frac{h}{T}\frac{L_{pp}}{B}\right)^{0.125}$$

The blockage factor is bounded to avoid invalid inverse trigonometric operations:

$$S_{safe} = \min\left(\max(S, 0), 0.999\right)$$

Then:

$$K_C = \left[2\sin\left(\frac{1}{3}\arcsin(1 - S_{safe})\right)\right]^{1.5}$$

and:

$$K_R = K_U\left(1 - \frac{h_T}{h}\right) + K_C\left(\frac{h_T}{h}\right)$$

Step 3: critical speed selection

The critical speed depends on channel type:

For unrestricted:

$$V_{cr} = C_U K_U$$

For canal:

$$V_{cr} = C_m K_C$$

For restricted:

$$V_{cr} = C_{mT} K_R$$

Then the speed ratio is limited:

$$\eta = \min\left(0.99, \frac{V_s}{V_{cr}}\right)$$

Step 4: form and speed multipliers

The code defines:

$$C_V = 8\eta^2\left[(\eta - 0.5)^4 + 0.0625\right]$$

$$C_F = \left(\frac{10C_B}{L_{pp}/B}\right)^2$$

$$K_{\Delta T} = 0.155\sqrt{\frac{h}{T}}$$

Step 5: bow and stern squat

The script computes:

$$S_{R,bow} = C_V C_F K_{\Delta T} T$$

$$S_{R,stern} = C_V K_{\Delta T} T$$

and adopts:

$$S_{Römisch} = \max(S_{R,bow}, S_{R,stern})$$

Interpretation

This formulation introduces geometry-dependent celerity control and a speed-to-critical-speed structure. It is more explicitly hydraulic than simpler concept methods.

9.3 Millward (1992)

The Millward method in the script calculates separate sinkage values at the forward and aft perpendiculars.

The code first limits the squared depth Froude number:

$$F_{safe}^2 = \min(F_{nh}^2, 1.05)$$

Forward sinkage:

$$S_{FP} = \left(15C_B\frac{T}{L_{pp}} - 0.55\right)\left(\frac{F_{safe}^2}{1 - 0.9F_{safe}^2}\right)\left(\frac{L_{pp}}{100}\right)$$

Aft sinkage:

$$S_{AP} = \left(61.7C_B\frac{T}{L_{pp}} - 0.6\right)\left(\frac{L_{pp}}{100}\right)\left(\frac{F_{safe}^2}{\sqrt{1 - F_{safe}^2}}\right)$$

with the square-root denominator guarded in the code as:

$$\sqrt{\max(0.001, 1 - F_{safe}^2)}$$

The adopted Millward squat is:

$$S_{Millward} = \max(S_{FP}, S_{AP})$$

Interpretation

Millward is especially sensitive to the combined effect of:

- ship fullness through C_B;
- relative draft T/L_{pp};
- depth criticality through F_{nh}.

Because the formula can become numerically aggressive near critical conditions, the script uses caps and guards to preserve stability.

9.4 Eryuzlu (1994)

The Eryuzlu method in the script includes an explicit width correction.

The width-to-beam ratio is:

$$\frac{W}{B}$$

The width correction is:

$$K_b = 1.0$$

for either unrestricted water or $W/B \geq 9.61$.

Otherwise:

$$K_b = \frac{3.1}{\sqrt{W/B}}$$

The squat formula implemented is:

$$S_{Eryuzlu} = 0.298 \left(\frac{h^2}{T}\right) F_{nT}^{2.289} \left(\frac{h}{T}\right)^{-2.972} K_b$$

This can be rewritten more compactly as:

$$S_{Eryuzlu} = 0.298\, h^2\, T^{-1}\, F_{nT}^{2.289} \left(\frac{h}{T}\right)^{-2.972} K_b$$

but the script effectively computes it as three factors multiplied together.

Interpretation

This method is strongly driven by:

- draft-based Froude scaling;
- relative depth h/T;
- lateral confinement through K_b.

9.5 Barrass B3 (2004)

The Barrass implementation is simple and widely used at concept or operational level.

Step 1: blockage multiplier

For unrestricted conditions or very small blockage:

$$K_{Barrass} = 1.0$$

if either:

$$\text{channel type} = \text{unrestricted}$$

or

$$S \leq 0.10$$

Otherwise:

$$K_{Barrass} = \max\left(1.0, \min\left(5.74S^{0.76}, 2.0\right)\right)$$

Thus the blockage multiplier is constrained to the interval $[1, 2]$.

Step 2: main squat equation

The maximum squat is then:

$$S_{Barrass} = \frac{K_{Barrass} C_B V_k^2}{100}$$

where V_k is in knots.

Step 3: opposite-end squat

The script also computes an opposite-end multiplier:

$$K_{oe} = 1 - 40(0.7 - C_B)^2$$

and then:

$$S_{oe} = K_{oe} S_{Barrass}$$

However, for the final six-method ranking, the code uses the main Barrass squat value $S_{Barrass}$, not S_{oe}.

Interpretation

This is a deliberately compact operational method. It is less structurally detailed than Römisch or Ankudinov, but it remains useful because of its simplicity and its wide practical diffusion.

9.6 Ankudinov / MARSIM (2009)

The Ankudinov section is the most composite squat model in the code. It separates midship sinkage from dynamic trim and then combines them.

Step 1: hull and Froude parameters

The script computes:

$$P_{Hu} = 1.7C_B \left(\frac{BT}{L_{pp}^2}\right) + 0.004C_B^2$$

$$P_{hT} = 1 + \frac{0.35}{(h/T)^2}$$

$$P_{Fnh} = F_{nh}^{(1.8+0.4F_{nh})}$$

Step 2: channel modifiers

For unrestricted water, the trench-ratio term is forced to zero:

$$\left(\frac{h_T}{h}\right)_{ank} = 0$$

Otherwise:

$$\left(\frac{h_T}{h}\right)_{ank} = \frac{h_T}{h}$$

Then:

$$S_h = C_B \left(\frac{S}{h/T}\right) \left(\frac{h_T}{h}\right)_{ank}$$

$$P_{Ch1} = 1 + 10S_h - 1.5(1 + S_h)\sqrt{S_h}$$

$$P_{Ch2} = \max(0, 1 - 5S_h)$$

Step 3: midship sinkage

With a fixed coefficient $K_{p,S} = 0.15$, the script defines:

$$S_m = (1 + K_{p,S}) P_{Hu} P_{Fnh} P_{hT} P_{Ch1}$$

Step 4: dynamic trim

The trim exponent is:

$$n_{Tr} = 2 + 0.8 \left(\frac{P_{Ch1}}{C_B} \right)$$

Additional fixed coefficients are set as:

$$K_{p,T} = 0.15, \quad K_{b,T} = 0.10, \quad K_{v,T} = 0.04, \quad K_{T1,T} = 0.0$$

Then:

$$K_{Tr} = C_B^{n_{Tr}} - (0.15 K_{p,S} + K_{p,T}) - (K_{b,T} + K_{v,T} + K_{T1,T})$$

The script next defines a modifier using the ratio h/T:

If $h/T \geq 1$:

$$P_{hT,trim} = 1 - e^{-1.2(h/T-1)}$$

Else:

$$P_{hT,trim} = 0$$

The dynamic trim is then:

$$Tr = -1.7 P_{Hu} P_{Fnh} P_{hT,trim} K_{Tr} P_{Ch2}$$

Step 5: total squat from sinkage and trim

The adopted Ankudinov squat is:

$$S_{Ankudinov} = L_{pp} \left(S_m + 0.5 |Tr| \right)$$

Interpretation

This structure is more physically layered than the simpler formulas because it distinguishes:

- a midship sinkage mechanism;
- a trim mechanism;
- hull-form sensitivity;
- finite-depth amplification;
- confinement modifiers.

---

10. Why the script adopts the average of the top three squat values

Once all six squat values are computed, the program sorts them in descending order and uses:

$$S_{adopted} = \frac{S_{(1)} + S_{(2)} + S_{(3)}}{3}$$

This deserves explicit interpretation.

If the code used only the single largest squat, the result could become excessively controlled by one method that is outside its best application range. If it used the mean of all six, the result might become too diluted by formulas known to underpredict for certain geometries or conditions. By averaging the three largest values, the calculator effectively says:

- keep the estimate conservative;
- do not allow the result to be dominated by only one equation;
- still retain the influence of the upper envelope.

In practical design terms, this is a **conservative consensus estimate**.

---

11. Density penalty

The program adds a density-related draft allowance using:

$$\Delta T_\rho = T\, 0.025 \left(\frac{C_B}{C_{WP}} \right)$$

This is a simplified Archimedean-type allowance representing the extra immersion expected when the water actually supporting the ship is less dense than the reference seawater condition implicit in the static draft.

Interpretation

The expression scales linearly with draft T. It also increases with fuller forms, since larger C_B relative to C_{WP} implies a stronger immersion increment for the same buoyancy deficit.

This term is intentionally simple. The code does not ask for actual water density, loading displacement at multiple drafts, or TPC curves. It uses a single operational allowance instead.

---

12. Wind-induced heel calculation

The wind heel model combines an aerodynamic lateral force with a hydrostatic restoring moment scale.

12.1 Displacement volume for restoring moment

The script converts the displacement estimate in metric tonnes into a volume estimate:

$$V_{disp,vol} = \frac{Disp_{MT} \times 1000}{1025}$$

where $1025\,\mathrm{kg/m^3}$ is the seawater density used in the conversion.

The restoring moment factor is then:

$$M_{rest} = \gamma_w V_{disp,vol} GM$$

This has the dimensional structure of a righting moment scale.

12.2 Aerodynamic coefficient build-up

The relative wind angle is converted to radians:

$$\theta_{WR,rad} = \theta_{WR} \frac{\pi}{180}$$

The code uses three harmonic terms. For each harmonic order $n = 1, 2, 3$, it computes a coefficient:

$$C_{Yn} = c_0 + c_1 \left(\frac{A_{VL}}{L_{pp}^2} \right) + c_2 \left(\frac{x_L}{L_{pp}} \right) + c_3 \left(\frac{L_{pp}}{B} \right) + c_4 \left(\frac{A_{VL}}{A_{VF}} \right)$$

Then the lateral aerodynamic coefficient is assembled as:

$$C_{Wy} = \sum_{n=1}^{3} C_{Yn} \sin(n\theta_{WR,rad})$$

The numerical coefficients $(c_0, c_1, c_2, c_3, c_4)$ are hard-coded in the script.

12.3 Lateral wind force and heeling moment

The lateral wind force is:

$$F_{Wy} = \frac{1}{2}\rho_a C_{Wy} A_{VL} V_{WR}^2$$

The lever arm is approximated by:

$$l_W = KG_W - \frac{T}{2}$$

Then the heeling moment is:

$$M_W = l_W F_{Wy}$$

and the heel angle is computed as:

$$\phi_W = \frac{M_W}{M_{rest}}$$

The code then converts this angle from radians-equivalent small-angle form into degrees for reporting:

$$\phi_{W,deg} = \phi_W \frac{180}{\pi}$$

Interpretation

This is essentially a **small-angle linearised static balance** between lateral aerodynamic heeling moment and hydrostatic restoring moment.

---

13. Turn-induced heel calculation

The turn-induced heel routine models centrifugal heeling under steady turn conditions and then multiplies it by a transient amplification factor.

13.1 Rudder angle in radians

$$\delta_{R,rad} = \delta_R \frac{\pi}{180}$$

13.2 Turning radius

The script computes:

$$R_C = \frac{L_{pp}}{K_R \delta_{R,rad}}$$

13.3 Centrifugal heel

The code evaluates a steady turn heel angle:

$$\phi_C = \frac{(KG - T/2)\, V_s^2}{g R_C GM}$$

Then it applies a transient multiplier:

$$\phi_R = C_\phi \, \phi_C$$

and reports the angle in degrees.

Implementation interpretation

The turning-heel branch uses the **manoeuvring turning index** entered by the user as K_R in the turning-radius expression,

$$R_C = \frac{L_{pp}}{K_R \, \delta_{R,rad}}$$

while the Römisch squat formulation uses its own internally derived hydraulic correction factor inside the critical-speed branch. These are distinct quantities in purpose and physical meaning, even though compact notation in the literature can make the symbols look similar across different formulations.

Accordingly, the turn-induced heel result should be interpreted as being controlled by:

- the user-entered turning index K_R;
- rudder angle δ_R;
- ship speed V_s;
- transverse stability through GM;
- vertical lever arm through $KG - T/2$; and
- gravity g.

This distinction matters because the turning-heel model is a manoeuvring-response model, whereas the Römisch factor is a confinement-sensitive hydraulic correction used only inside one squat method.

---

14. Conversion of heel angle into vertical bilge penetration

The script adds wind and turning heel angles:

$$\phi_{total} = \phi_{W,deg} + \phi_{R,deg}$$

Then it converts the total heel into an equivalent downward bilge penetration:

$$Z_{heel} = F_K \left(\frac{B}{2}\right) \sin\left(\phi_{total} \frac{\pi}{180}\right)$$

where F_K is a ship-type factor.

Interpretation

This is a geometric projection of the half-beam into a vertical sinkage at the low side of the vessel, scaled by an empirical correction factor. It does not perform a full cross-curve or KN-based large-angle stability calculation. It is a compact operational allowance model.

---

15. Wave-induced vertical motion allowance

The wave module is one of the most distinctive parts of the script because it combines three different simplified methods and then averages them.

15.1 Wavelength from external nonlinear function

Before computing the three wave-allowance methods, the script obtains a wavelength from the external solver call:

$$\lambda_w = L(H_s, T_w, h, V_s)$$

where `function.L` is imported from `function.py`.

The wavelength is therefore **not** calculated by a simple local linear-dispersion expression inside `navigation-calculator.py`. Instead, it is delegated to a standalone nonlinear wavelength engine implemented in `function.py`, which exposes both an importable API and a direct command-line interface. That

external file describes itself as a **Fenton stream-function / Fourier finite-depth steady-wave solver** using period input and an Eulerian-current criterion.

Why this is more accurate than using only linear wavelength

A purely linear wavelength calculation is based on infinitesimal-wave theory. In practical terms, that means it assumes the wave amplitude is sufficiently small that the free surface remains sinusoidal, the kinematics remain first-order, and the wavelength depends only on period and depth through the linear dispersion relation. That is often acceptable for quick screening, but it becomes progressively less representative when any of the following are important:

- finite wave height relative to depth or wavelength;
- shallow or intermediate water where nonlinearity matters;
- interaction between wave propagation and a background velocity/current-type kinematic term;
- operational situations where wavelength is reused inside motion or UKC allowance formulas that are sensitive to wave slope.

The Fenton-based solver is more accurate for this application because it includes effects that the linear approximation omits.

15.1.1 Finite-amplitude nonlinearity

Linear theory treats the free surface as a small perturbation and therefore cannot capture the amplitude-dependent change in celerity and wavelength that appears as waves become steeper. The Fenton stream-function formulation instead solves a nonlinear steady periodic wave problem. In consequence, the returned wavelength is not just the infinitesimal-theory wavelength at the same period and depth; it is the wavelength associated with the nonlinear free-surface solution compatible with the specified inputs.

15.1.2 Finite-depth consistency beyond first order

Although linear theory can handle finite depth, it still does so under small-amplitude assumptions. The Fenton solver remains finite-depth but also preserves nonlinear free-surface geometry and velocity-field structure. For UKC work, this matters because many operational cases occur exactly in the shallow-to-intermediate-depth range where confinement, hydrodynamic squat, and wave-induced motions are all relevant at the same time.

15.1.3 Sensitivity to the fourth solver argument used by the UKC script

The external solver accepts a fourth kinematic argument, and the main UKC script passes the value used in its wavelength call directly as the fourth parameter of `function.L(H_s, T_w, h, V_s)`. This means the wavelength used

downstream is not merely a depth-only linear estimate; it is the output of the same nonlinear solver core under the full argument set supplied by the calculator.

15.1.4 Numerical consistency across the toolchain

Another practical advantage is consistency. The wavelength engine exists as an external standalone module that can also be called independently from other scripts. That means:

- the GUI/CLI UKC calculator and external tools can reuse the same wavelength formulation;
- the wave allowance section is traceable to a single solver implementation;
- future validation of wavelength values can be performed directly against `function.py` without re-deriving formulas inside the UKC script.

15.1.5 Engineering implication inside this calculator

The wavelength enters the Japanese Z3-type method through the effective slope term,

$$\Phi = 360 \left(\frac{0.35 H_s}{\lambda_w} \right) \sin \left(\psi \frac{\pi}{180} \right)$$

so any improvement in λ_w propagates directly into the roll-related wave allowance. If wavelength were underestimated or overestimated by relying only on a simplified linear model, the downstream vertical-motion allowance would inherit that bias.

For that reason, using the external Fenton wavelength is more appropriate than using only the linear wavelength whenever the objective is not just generic wave description, but a **clearance-critical operational allowance** embedded in a larger UKC budget.

15.2 Wave Method 1: bounded trigonometric worst-case allowance

The script first defines two motion surrogates:

$$Z_{\phi,1} = 0.044B$$

$$Z_{\theta,1} = 0.0087L_{pp}$$

These are added:

$$Z_{sum,1} = Z_{\phi,1} + Z_{\theta,1}$$

Then the result is capped by twice the significant wave height:

$$z_{max,1} = \min(Z_{sum,1}, 2H_s)$$

Interpretation

This method is intentionally conservative and geometric. It acts as a bounded surrogate for combined roll- and pitch-related vertical excursion.

15.3 Wave Method 2: Japanese Z3-type bilge sinkage approach

The second method uses the externally obtained wavelength.

If $\lambda_w > 0$, the script computes:

$$\Phi = 360 \left(\frac{0.35 H_s}{\lambda_w} \right) \sin \left(\psi \frac{\pi}{180} \right)$$

Otherwise:

$$\Phi = 0$$

Then the maximum roll angle surrogate is:

$$\phi_{max} = \mu \gamma \Phi$$

The unbounded Z3-type allowance is:

$$Z_{3,unbounded} = 0.7 \left(\frac{H_s}{2} \right) + \left(\frac{B}{2} \right) \sin \left(\phi_{max} \frac{\pi}{180} \right)$$

The code then applies a hard physical cap:

$$Z_3 = \min(Z_{3,unbounded}, 2H_s)$$

A chart-based sinkage placeholder exists in the script as:

$$z_{2,chart} = 0$$

Therefore the adopted value for Method 2 is:

$$z_{max,2} = \max(z_{2,chart}, Z_3) = Z_3$$

in the present implementation.

Interpretation

This method tries to account for wave-slope forcing and amplified roll response while preventing unrealistic blow-up when wavelength becomes very short relative to ship dimensions.

15.4 Wave Method 3: ROM 3.1 probabilistic allowance

The third wave method is a multi-factor probabilistic product.

Factor C_1

The script defines:

$$C_1 = 0.707\sqrt{\ln\left(\frac{N_w}{-\ln(1-P_m)}\right)}$$

This is a probabilistic amplification factor based on the number of encountered waves and exceedance probability.

Factor C_2

The code uses a tabulated matrix and bilinear-style interpolation based on L_{pp} and H_s. In implementation terms, this is done by repeated one-dimensional interpolation, but conceptually it acts as a two-parameter interpolated coefficient:

$$C_2 = C_2(L_{pp}, H_s)$$

Factor C_3

The loading correction is interpolated from loading percentage:

$$C_3 = C_3(\text{load\%})$$

specifically between 50% and 90% load.

Factor C_4

The depth-Froude correction is:

$$C_4 = C_4(F_{nh})$$

interpolated between $F_{nh} = 0.05$ and $F_{nh} = 0.25$.

Factor C_5

The relative-depth correction is:

$$C_5 = C_5\left(\frac{h}{T}\right)$$

interpolated between $h/T = 1.05$ and $h/T = 1.50$.

Factor C_6

The angle correction uses a normalised encounter angle. The script first folds the wave angle into the 0° to 90° range:

$$\psi_{norm} = \psi \bmod 180$$

and if $\psi_{norm} > 90°$ it uses:

$$\psi_{norm} = 180° - \psi_{norm}$$

Then the angle factor is interpolated from the sequence:

- $(0°, 1.0)$
- $(15°, 1.0)$
- $(35°, 1.4)$
- $(90°, 1.7)$

Thus conceptually:

$$C_6 = C_6(\psi_{norm})$$

Final ROM 3.1 wave allowance

The unbounded value is:

$$z_{max,3,unbounded} = H_s C_1 C_2 C_3 C_4 C_5 C_6$$

The code caps it by twice the significant wave height:

$$z_{max,3} = \min(z_{max,3,unbounded}, 2H_s)$$

15.5 Adopted wave allowance

After computing the three wave allowances, the script adopts their arithmetic mean:

$$Z_{wave} = \frac{z_{max,1} + z_{max,2} + z_{max,3}}{3}$$

This mirrors the overall philosophy already used in the squat block: combine multiple methods rather than trusting a single one.

---

16. Final UKC summation logic

Once all submodels are complete, the script evaluates:

$$UKC_{gross} = h - T$$

$$\Delta Z_{total} = S_{adopted} + \Delta T_{\rho} + Z_{heel} + Z_{wave}$$

$$UKC_{net} = UKC_{gross} - \Delta Z_{total}$$

$$D_{eff} = h - \Delta Z_{total}$$

$$R_{DT} = \frac{D_{eff}}{T}$$

These are the program's primary outputs.

Interpretation of the two main final indicators

The script reports both:

- **net UKC**, which is the residual physical gap between keel and seabed after allowances;
- **effective depth-to-draft ratio**, which is a normalised clearance indicator.

The ratio is especially useful because it scales the result with vessel draft and allows quick safety screening across ships of different size.

---

17. Depth-search solver for minimum safe depth

If the computed ratio is below the acceptance criterion, the script enters a second-stage process that searches for a larger depth satisfying the safety requirement.

17.1 Trigger condition

The internal solver is activated when:

$$\frac{D_{eff}}{T} < 1.10$$

17.2 Re-evaluation at trial depths

The code defines an internal function `evaluate_at_depth(h_test)` that recomputes, at a new candidate depth:

- the six squat formulas;
- the top-three adopted squat;
- the wavelength-dependent wave allowance;
- the total dynamic sinkage;
- the effective depth ratio.

Importantly, the density penalty and bilge penalty remain unchanged in that nested routine because they do not depend explicitly on the trial depth in the current implementation.

17.3 Search objective

The solver seeks the smallest depth on its internal depth grid that satisfies the criterion with a small internal margin. Operationally, the intent is to return the minimum nominal depth that produces an acceptable effective depth ratio.

This makes the calculator not only an evaluator of a given depth, but also a **required-depth estimator**.

---

18. Numerical protection and boundedness measures used in the code

A notable feature of the implementation is that it contains several numerical guards. These are essential because squat and wave formulas often become

unstable near critical conditions.

18.1 Square-root protections

Whenever a formula contains terms such as:

$$\sqrt{1 - F^2}$$

the script often replaces the inner term by:

$$\max(0.001, 1 - F^2)$$

This avoids undefined operations when the theoretical denominator becomes zero or negative.

18.2 Caps on speed ratio and Froude expressions

The Römisch speed ratio is limited by:

$$\eta \leq 0.99$$

The Millward routine caps:

$$F_{safe}^2 \leq 1.05$$

These caps prevent near-singular growth in the formulas.

18.3 Bounding of blockage for inverse trigonometric evaluation

The Römisch implementation uses:

$$S_{safe} = \min(\max(S, 0), 0.999)$$

before evaluating $\arcsin(1 - S_{safe})$.

18.4 Hard caps on wave allowances

All three wave methods are limited, directly or indirectly, by:

$$2H_s$$

This is a deliberate physical bound intended to suppress mathematically large but operationally implausible values.

19. Implementation mapping to `navigation-calculator.py`

This section avoids repeating the equations already developed in Sections 6 to 18 and instead states how those equations are used in the program.

19.1 Formulas effectively applied by the script

All governing equations described earlier in this README are **actually executed** in `navigation-calculator.py`. In practical terms, the script performs the following blocks in sequence:

- reconstruct ship-type hydrostatic surrogates from the embedded vessel table;
- derive A_S, A_C, W_{Top}, blockage S, F_{nh}, and F_{nT};
- evaluate the six squat methods;
- rank the squat outputs and adopt the average of the three largest values;
- compute density, wind-heel, turning-heel, and bilge-penetration penalties;
- obtain wavelength from `function.L(H_s, T_w, h, V_s)`;
- evaluate the three wave-allowance methods and adopt their arithmetic mean;
- compute gross UKC, total dynamic sinkage, net UKC, effective depth, and effective depth ratio;
- re-evaluate the same depth-dependent model inside the internal minimum-depth solver whenever the initial case is below the acceptance threshold.

The important implementation point is that the depth-search routine does **not** switch to a simplified model. It recomputes the same squat and wave logic at each trial depth.

19.2 Formulas computed but not decisive in the final ranking

The Barrass block also computes an opposite-end squat estimate,

$$S_{oe} = K_{oe} S_{Barrass}$$

but that value is not one of the six competitors used in the top-three squat selection. It is therefore informative, not decisive, in the final adopted squat.

19.3 External dependency in the calculation chain

The wavelength is obtained from the external module:

$$\lambda_w = L(H_s, T_w, h, V_s)$$

Accordingly, any revision to `function.py` will propagate directly into the wave-motion block and into the depth-search solver.

---

20. Engineering interpretation of each major allowance

To understand the output properly, it is useful to interpret the meaning of the four main dynamic terms.

20.1 Adopted squat

This is the hydrodynamic sinkage-and-trim allowance produced by the six-method squat tournament. In many practical transits it is the dominant loss term because it grows rapidly with speed and is intensified by shallow depth and lateral confinement.

20.2 Density penalty

This is a buoyancy correction, not a speed effect. It accounts for the possibility that the ship floats deeper than the nominal seawater reference draft when water density is lower.

20.3 Heel allowance

This is the low-side draft increase caused by combined wind-induced and turn-induced heeling, projected geometrically into a bilge-penetration depth.

20.4 Wave allowance

This is the reserved depth for vertical ship motions in waves. In the present calculator it is not a ship-specific seakeeping solution; it is a practical average of three simplified allowance methods.

---

21. Limits of applicability and modelling cautions

This document explains the implemented model. It does not claim universal validity. Several limitations are built into the method chain.

21.1 Empirical nature of squat formulas

Most squat methods used here are empirical or semi-empirical. Their reliability depends on how closely the ship, depth regime, and channel geometry resemble the conditions from which the original formulas were derived.

21.2 Simplified hydrostatics

The program infers GM, KG, windage areas, and related properties from vessel type instead of using an actual hydrostatic booklet or loading computer export. That is suitable for screening and comparative studies, but not for final approval of a critical transit.

21.3 Additive conservative combination

All allowances are added directly. This is conservative and easy to audit, but it ignores statistical dependence and phase relationships between different motions and forcing mechanisms.

21.4 Wave model simplification

The wave section uses compact allowance methods rather than a full RAO-based, spectral, or time-domain seakeeping model. It should therefore be read as a navigation-planning reserve, not a full ship-motion simulation.

21.5 Turning-heel parameter interpretation

The turn-induced heel routine compresses a complex manoeuvring event into a deterministic turning-radius and heel estimate driven mainly by K_R, δ_R, V_s, KG, T, and GM. It is therefore best interpreted as a practical allowance model.

---

22. Recommended reading of the output report

A useful reading sequence is:

1. confirm the ship particulars and water depth;
2. inspect blockage S, F_{nh}, and h/T;
3. compare the six squat outputs and identify the governing methods;
4. inspect the adopted squat, density, heel, and wave allowances separately;
5. read the final net UKC and the effective depth ratio;
6. if required, read the diagnosis and the recommended minimum nominal depth.

The subsections below reproduce the main embedded tables and report structures in GitHub-flavoured markdown.

22.1 Embedded vessel-type coefficient table used by the code

Vessel type	C_B	$(GM/T)_{type}$	F_K	a_f	b_f	a_l	b_l
cargo	0.72	0.20	0.83	0.592	0.666	3.213	0.616
bulk	0.85	0.35	0.88	8.787	0.370	16.518	0.425
container	0.60	0.10	0.78	1.369	0.609	2.614	0.703

Vessel type	C_B	$(GM/T)_{type}$	F_K	a_f	b_f	a_l	b_l
tanker	0.85	0.35	0.88	2.946	0.474	3.598	0.558
roro	0.65	0.15	0.80	10.697	0.435	28.411	0.464
passenger	0.60	0.15	0.80	8.842	0.426	3.888	0.680
ferry	0.60	0.15	0.80	5.340	0.473	3.666	0.674
gas	0.75	0.30	0.83	2.649	0.553	5.074	0.613

These coefficients drive the internal hydrostatic reconstruction, bilge factor, and windage-area regressions.

22.2 Default input table used by the GUI wrapper

Key	Default value	Meaning
`v_type`	`bulk`	Vessel type
`L_pp`	`213.0`	Length between perpendiculars [m]
`B`	`32.3`	Maximum beam [m]
`T`	`12.8`	Static draft [m]
`tonnage`	`60000`	Reference DWT/GT
`load_pct`	`100.0`	Loading condition [%]
`V_s`	`3.86`	Transit speed [m/s]
`h`	`14.0`	Nominal water depth [m]
`channel_type`	`restricted`	Channel class
`W/B ratio`	`3.5`	Width to beam ratio
`W`	`113.05`	Derived channel bottom width [m]
`h_T`	`0.0`	Trench height [m]
`n_bank`	`7.0`	Inverse bank slope
`V_WR`	`10.0`	Relative wind speed [m/s]
`theta_WR`	`90.0`	Relative wind angle [deg]
`K_R`	`0.5`	Turning index [-]
`delta_R`	`15.0`	Rudder angle [deg]
`C_phi`	`1.5`	Transient heel coefficient [-]
`H_s`	`0.5`	Significant wave height [m]
`T_wave`	`3.0`	Wave period [s]
`wave_angle`	`90.0`	Wave incidence angle [deg]
`mu_roll`	`3.0`	Roll magnification factor [-]
`gamma_slope`	`0.8`	Effective slope coefficient [-]
`N_w`	`500`	Number of waves encountered [-]
`P_m`	`0.00115`	Probability of exceedance [-]

22.3 Exact CLI input order reproduced by the GUI

The GUI feeds the CLI engine in the following order:

Order	Key	Prompt meaning
1	`v_type`	Vessel type
2	`L_pp`	Length between perpendiculars
3	`B`	Maximum beam
4	`T`	Static draft
5	`tonnage`	Tonnage (DWT or GT)
6	`load_pct`	Loading condition
7	`V_s`	Transit speed
8	`h`	Nominal water depth
9	`channel_type`	Channel type
10	`W`	Bottom width
11	`h_T`	Trench height
12	`n_bank`	Inverse bank slope
13	`V_WR`	Relative wind speed
14	`theta_WR`	Relative wind angle
15	`K_R`	Non-dimensional index of turning
16	`delta_R`	Rudder angle
17	`C_phi`	Transient heel coefficient
18	`H_s`	Significant wave height
19	`T_wave`	Wave period
20	`wave_angle`	Wave incidence angle
21	`mu_roll`	Roll magnification factor
22	`gamma_slope`	Effective wave slope coefficient
23	`N_w`	Number of waves encountered
24	`P_m`	Probability of exceedance

22.4 GUI tab tables

22.4.1 Vessel tab

Field key	GUI label
`v_type`	Vessel type
`L_pp`	Length between perpendiculars, L_pp (m)
`B`	Maximum beam, B (m)
`T`	Static draft, T (m)
`tonnage`	Tonnage (DWT or GT)
`load_pct`	Vessel loading condition (%)
`V_s`	Transit speed, V_s (m/s)

22.4.2 Channel tab

Field key	GUI label	Note
`h`	Nominal water depth, h (m)	
`channel_type`	Channel type	unrestricted / restricted / canal
`W/B ratio`	Channel width ratio, W/B	user-editable
`W`	Channel bottom width, W (m)	derived from $B \times (W/B)$
`h_T`	Trench height, h_T (m)	0 for canal or unrestricted
`n_bank`	Inverse bank slope (run/rise), n	

22.4.3 Metocean tab

Field key	GUI label
`V_WR`	Relative wind speed, V_WR (m/s)
`theta_WR`	Relative wind angle, theta_WR (deg)
`H_s`	Significant wave height, H_s (m)
`T_wave`	Wave period, T_w (s)
`wave_angle`	Wave incidence angle, psi (deg)
`mu_roll`	Roll magnification factor, mu
`gamma_slope`	Effective wave slope coefficient, gamma
`N_w`	Number of waves encountered, N_w
`P_m`	Probability of exceedance, P_m

22.4.4 Manoeuvre tab

Field key	GUI label
`K_R`	Non-dimensional index of turning, K_R
`delta_R`	Rudder angle, delta_R (deg)
`C_phi`	Transient heel coefficient, C_phi

22.4.5 Run / Report tab

GUI control	Purpose
optional output path box	saves a copy of the extracted report
`Run calculation`	executes the same engine used by CLI mode
`Copy to clipboard`	copies the report text
`Save As…`	exports the displayed report
`Reset defaults`	restores GUI defaults
output panel	displays the extracted fixed-width report

22.5 Report Table 1 — Input Parameters

Report section	Main content
`[A] Vessel`	ship type, L_{pp}, B, T, tonnage, loading condition
`[B] Channel & transit`	h, channel type, W, h_T, n
`[C] Kinematics & metocean`	wind, turning, wave, and wavelength inputs/results

22.6 Report Table 2 — Derived Hydrostatics and Kinematics

Report section	Main content
`[A] Hydrostatics & displacement`	C_B, C_{WP}, ∇, Δ
`[B] Stability & aerodynamics`	KB, BM, KG, GM, A_{VF}, A_{VL}, KG_W
`[C] Kinematics & channel`	V_s, V_k, F_{nh}, F_{nT}, A_S, A_C, S

22.7 Report Table 3 — Hydrodynamic Squat (PIANC Analysis)

Report block	Meaning
ICORELS (1980)	volumetric shallow-water squat estimate
Römisch (1989)	bow/stern form-sensitive squat estimate
Millward (1992)	forward/aft sinkage pair with governing maximum
Eryuzlu (1994)	draft-Froude and width-correction method
Barrass B3 (2004)	blockage- and speed-based concept formula
Ankudinov MARSIM (2009)	midpoint-sinkage plus trim formulation
tournament ranking	six methods sorted from largest to smallest
adopted squat line	average of the three largest values

22.8 Report Table 4 — Environmental Sinkages and Heel Penalties

Report block	Meaning
water density loss penalty	density-related draft increment
asymmetric dynamic heel	wind heel, turn heel, combined heel, and bilge penetration

22.9 Report Table 5 — Wave-Induced Vertical Motions

Report block	Meaning
Method 1	bounded roll-plus-pitch reserve
Method 2	wavelength-driven Japanese Z3-type allowance
Method 3	ROM 3.1 probabilistic allowance

Report block	Meaning
adopted wave line	arithmetic mean of the three wave methods

22.10 Report Table 6 — Final UKC summation

Report row	Meaning
Gross UKC `(h - T)`	static geometric clearance [m]
Adopted Hydrodynamic Squat	selected squat allowance [m]
Adopted Wave Allowance	selected wave allowance [m]
Water Density Penalty	density allowance [m]
Dynamic Bilge Heel Penalty	heel allowance [m]
Total Dynamic Sinkage `(Delta T_dyn)`	sum of all penalties [m]
FINAL NET UKC	remaining under-keel clearance [m]
Effective Water Depth `(D_eff)`	depth remaining after allowances [m]
Effective Depth to Draft Ratio `(D_eff / T)`	final acceptance ratio [-]

22.11 Diagnostic and recommendation lines

Diagnostic output	Meaning
`CRITICAL / UNSAFE`	below the minimum ratio threshold
`MARGINAL (CALM WATERS ONLY)`	ratio satisfies minimum but limited reserve
`STANDARD / SAFE`	acceptable under the internal classification
`HIGH CLEARANCE`	comparatively comfortable reserve
`RECOMMENDATION`	no added depth required or minimum depth stated

---

23. Conceptual summary of the entire computational chain

The complete logic of the calculator can be read as the following ordered sequence.

Step 1: read inputs

Read vessel particulars, speed, channel geometry, wind, manoeuvre inputs, and wave inputs.

Step 2: reconstruct ship coefficients

Recover ship-type surrogate properties such as C_B, GM, windage areas, and bilge factor from the embedded vessel table.

Step 3: derive flow and confinement variables

Build blockage, hydraulic width/area surrogates, and the Froude numbers required by the squat and wave methods.

Step 4: evaluate the squat tournament

Run the six squat models, sort the outputs, and keep the three highest values for conservative averaging.

Step 5: compute non-squat penalties

Add density allowance, wind heel, turning heel, bilge penetration, and the three wave-motion allowances.

Step 6: sum the UKC budget

Subtract the adopted allowances from water depth relative to static draft and compute net UKC plus the effective depth ratio.

Step 7: classify the result

Assign a diagnosis according to the resulting depth-to-draft ratio.

Step 8: search for a compliant nominal depth if needed

If the case is unsafe, re-evaluate the same depth-dependent physics at increasing trial depths until a verified minimum compliant depth is found.

---

24. What this README is intended to document

This README is intentionally focused on the **actual computational content** of `navigation-calculator.py`. It is meant to serve both as:

1. a theoretical explanation of the implemented UKC model; and
2. a technical guide for anyone maintaining, validating, or extending the script.

The document therefore keeps the formulas that are genuinely used by the calculator, while avoiding later repetition of the same equations when a cross-reference is sufficient.

---

25. Final practical interpretation

In practical terms, the calculator is a **vertical-clearance budget**. Section 16 contains the governing final summation. Every earlier section exists only to define one of the terms in that summation.

The hydrodynamic block estimates sinkage and trim while underway. The density block adjusts static flotation. The heel block converts external and manoeuvring moments into low-side draft increase. The wave block reserves motion space in a simplified but operationally conservative way. The final ratio check then answers the operational question: *is the available effective depth still sufficient for the entered draft?*

---

26. Python environment, installation, and standalone executable build

The calculator depends on the main navigation script and the external nonlinear wavelength module. The practical minimum environment is:

Component	Purpose
Python 3	runtime interpreter
`numpy`	numerical support expected by `function.py`
`numba`	optional acceleration expected by `function.py`
`tkinter`	GUI layer used by `navigation-calculator.py`
`pyinstaller`	optional packaging tool for standalone executables

26.1 Run from source without a virtual environment

A virtual environment is **not required**. On a normal system-wide or per-user Python installation, install the needed packages directly into the user site-packages area:

```
python -m pip install --user --upgrade pip
python -m pip install --user numpy numba pyinstaller
```

`tkinter` is normally provided by the Python distribution itself rather than by `pip`. On Windows it is usually present already. On Linux it may need to be installed through the operating-system package manager if your Python build does not include Tk.

26.2 File layout expected by the calculator

For source execution, keep at least these files in the same working directory:

File	Role
`navigation-calculator.py`	main UKC calculator, GUI by default, CLI with `--cli`
`function.py`	external nonlinear wavelength engine
`README.md`	technical and operational documentation

26.3 Build a standalone executable without creating a virtual environment

For most users the simplest packaging route is PyInstaller.

Windows GUI executable

```
python -m PyInstaller --noconfirm --clean --onefile --windowed ^
--name navigation-calculator navigation-calculator.py
```

This creates a GUI-oriented executable in the `dist` folder.

Windows or Linux console executable

```
python -m PyInstaller --noconfirm --clean --onefile --console ^
--name navigation-calculator-cli navigation-calculator.py
```

This creates a console-oriented executable that is convenient when you want to launch the program with `--cli` and keep the terminal visible.

Notes on packaging

- Because `navigation-calculator.py` imports `function.py` directly, PyInstaller should detect that module automatically in normal cases.
- The packaged executable will appear inside the `dist` directory created by PyInstaller.
- A one-file executable bundles Python code, but GUI back-end libraries such as Tk may still depend on normal system support, especially on Linux.
- If you want only source execution, none of the PyInstaller steps are needed.

---

27. Script usage summary

The project exposes two user-facing Python entry points:

Script	Main purpose
`navigation-calculator.py`	main UKC calculator, CLI + GUI wrapper
`function.py`	standalone nonlinear wavelength calculation

27.1 `function.py` usage

`function.py` can be used both as an importable library and as a direct command-line tool.

Import usage

```
from function import L

lambda_w = L(H, T, d, U)
print(lambda_w)
```

The returned value is the wavelength in metres.

Command-line usage with positional arguments

```
python function.py 3 9 5 1
```

This prints a line of the form:

```
L = ... m
```

Interactive usage

```
python function.py
```

If run without arguments, the script prompts for wave height `H`, wave period `T`, water depth `d`, and current `U`.

---

28. `navigation-calculator.py` CLI usage

When launched in CLI mode, the script runs the UKC calculator in prompt/response style and writes the final fixed-width report both to the console and to `output.txt` in the working folder.

28.1 Explicit CLI launch

```
python navigation-calculator.py --cli
```

This is the authoritative way to obtain prompt-based CLI behaviour.

28.2 Plain launch from a terminal

```
python navigation-calculator.py
```

In the current implementation, plain launch still goes to the **GUI path by default**. Therefore use `--cli` whenever you explicitly want console prompting.

28.3 CLI outputs

Output	Description
console report	full framed PIANC UKC report
`output.txt`	saved text copy of the same report
success line	absolute path of the exported report file

---

29. `navigation-calculator.py` GUI usage

The same file can also open a Tkinter graphical interface.

29.1 Standard GUI launch

```
python navigation-calculator.py
```

GUI is the default execution path unless `--cli` is supplied.

29.2 Optional explicit GUI flag

```
python navigation-calculator.py --gui
```

The current dispatch logic does not require `--gui`; any invocation that does not include `--cli` falls through to GUI mode.

29.3 GUI structure

The GUI is organised into five tabs plus an output area inside the Run / Report tab.

GUI area	Purpose
Vessel tab	ship particulars and transit speed
Channel tab	depth, confinement, W/B ratio, and derived bottom width
Metocean tab	wind and wave inputs
Manoeuvre tab	turning inputs
Run / Report tab	execute calculation, review output, copy/export results

29.4 Practical GUI workflow

1. Enter or revise vessel particulars.
2. Confirm water depth and channel type.
3. Enter the W/B ratio if you want the GUI to derive channel bottom width automatically.
4. Enter metocean conditions.
5. Enter manoeuvre inputs.

6. Run the calculation and review the report in the output pane.
7. Optionally export the report to a text file.

29.5 Important GUI behaviour

- The GUI is only a presentation wrapper around the same `main()` engine used by CLI mode.
- The channel tab derives `W` automatically from beam and the entered W/B ratio.
- The output font is auto-fitted so the 100-character report width stays aligned in the text panel.
- Even when used through the GUI, the underlying engine still writes `output.txt` in the working folder.

---

30. Practical command examples

30.1 Run the full calculator in CLI mode

```
python navigation-calculator.py --cli
```

30.2 Run the full calculator in GUI mode

```
python navigation-calculator.py
```

30.3 Build a one-file GUI executable

```
python -m PyInstaller --noconfirm --clean --onefile --windowed ^
--name navigation-calculator navigation-calculator.py
```

30.4 Compute only nonlinear wavelength from the auxiliary solver

```
python function.py 3 9 5 1
```

30.5 Reuse the wavelength solver in another Python program

```
from function import L

lambda_w = L(3.0, 9.0, 5.0, 1.0)
print(f"lambda_w = {lambda_w:.6f} m")
```